

RCC Onboarding - Part 1

Concepts, secure access, remote editing, data transfer, and the first Slurm job

IKIM RCC documentation proposal

11 July 2026

Contents

About Part 1 and essential vocabulary

Information that must be completed before publication

Cluster architecture and the local/remote mental model

Accounts, authorisation, and data protection

Local software: OpenSSH and Visual Studio Code

SSH, keys, passphrases, and server identity

SSH configuration and the gateway route

Terminal connection and verification

VS Code Remote - SSH and remote editing

Storage, browser transfer, and checksums

Slurm, batch scripts, resources, and the first job

Connection persistence and tmux

Layered troubleshooting

Completion checklist

What comes in Part 2

About Part 1

This tutorial is for researchers and students who have not previously worked with a Linux cluster, SSH, Visual Studio Code, or Slurm.

By the end of Part 1, you will be able to:

install the required software on Windows or macOS;

create and safely handle an SSH key;

connect to the RCC cluster from a terminal and from Visual Studio Code;

transfer a small test file through the RCC web transfer service; and

submit, monitor, and inspect your first Slurm batch job.

Part 1 does **not** install scientific software or introduce Snakemake. Part 2 introduces reproducible workflows and scientific software. Part 3 explains performance and efficient data movement. Part 4 introduces Apptainer containers.

How to use this guide

This is not a list of commands to copy without context. Each major section follows the same pattern:

Concept: what the component is and which problem it solves.

Location: whether it is on your laptop, a gateway, a submission host, shared storage, or a compute node.

Action: the exact steps to perform.

Verification: a command or visible result that confirms success.

Failure boundary: the point at which you should stop and ask for support rather than guessing.

Read the concept paragraphs even when the commands appear familiar. Most cluster mistakes happen because a correct command is run on the wrong computer, in the wrong directory, or outside Slurm.

Essential vocabulary

Local: your Windows or macOS computer.

Remote: a computer in the RCC environment.

Terminal: a window that displays text and lets you type commands. PowerShell and macOS Terminal are local terminals; the VS Code terminal after connection is remote.

Shell: the program that reads commands in a terminal. RCC examples use Bash.

Command: an instruction executed by the shell, such as `pwd` or `sbatch`.

Path: the address of a file or directory, such as `/projects/example/data.csv`.

Process: a running program. Every command starts one or more processes.

Node: one computer belonging to the cluster.

Job: a unit of work submitted to Slurm with requested resources.

Cluster: multiple computers, storage systems, and network services managed as one research-computing environment.

Resource manager: software that decides when and where jobs run. RCC uses Slurm.

Four questions to ask before every command

Before pressing Enter, ask:

Which computer am I controlling? Your laptop, a login gateway, a submission host, or a compute node?

Where are the files? Local disk, home directory, project storage, or temporary node-local storage?

Where will the process run? Directly in the current shell, or as a Slurm job?

What happens if the connection closes? Does the process stop, continue under Slurm, or remain inside a tmux session?

These four questions form the mental model for both parts of the tutorial.

Information that must be completed before publication

Administrator action required The values below are specific to the RCC installation. Replace every value marked **ADMIN** before publishing this page. Do not ask users to guess these values.

SSH login gateway: login1.ikim.uk-essen.de

Slurm submission host: shellhost.ikim.uk-essen.de

Local SSH name used in this tutorial: rcc

RCC web transfer address: [ADMIN: insert the web transfer address]

RCC transfer collection: [ADMIN: insert the exact collection name]

Login identity provider: [ADMIN: state how users sign in to the transfer service]

VPN requirement: [ADMIN: state whether and when VPN is required]

RCC support contact: [ADMIN: insert support email or ticket address]

Login-gateway ED25519 fingerprint: [ADMIN: insert verified fingerprint]

Submission-host ED25519 fingerprint(s): [ADMIN: insert every valid fingerprint, or document the SSH host certificate authority]

Your project coordinator must also give you:

your RCC username;

your project or group name;

the exact directory in which you may store project data; and

confirmation that your public SSH key has been activated.

1. Understand the basic layout

Why a cluster exists

A laptop is designed for one person and normally has a small number of CPU cores, limited memory, and one local disk. Research analyses may need more memory, more processors, specialist hardware, shared datasets, or uninterrupted execution over many hours. A cluster combines many computers and shared services so that users can request the capacity needed by each task.

The RCC cluster is not one large personal computer. It consists of several systems with different purposes. Separating these roles protects the service and allows many researchers to work at the same time.

Your laptop

|
| SSH
v

Login gateway

|
| ProxyJump
v

Slurm submission host

|
| sbatch / queue
v

Compute node selected by Slurm

The login gateway authenticates your connection. It is comparable to a controlled entrance, not a workplace for analysis. The submission host is where you edit small text files, prepare workflows, submit jobs, and inspect their status. Compute-intensive work must run on compute nodes allocated by Slurm.

The systems have different responsibilities:

Your laptop: provides the user interface and stores the private SSH key. Use it for VS Code and a local terminal. Do not keep the only copy of important project data there.

Login gateway: provides the controlled entry to RCC. Use it for authentication and forwarding, not for analysis or large transfers.

Submission host: is the place for editing, Git, Slurm commands, and workflow coordination. Do not run scientific programs directly there.

Compute node: executes programs launched by Slurm. Beginners do not select a compute node manually.

Shared storage: holds approved project inputs and generated outputs. Do not store credentials or unapproved identifying data there.

The prompt tells you where you are - but verify it

A shell prompt often contains a host name, but prompts can be customised and should not be treated as proof. Use these commands:

```
whoami
hostname
pwd
```

whoami reports the remote account.

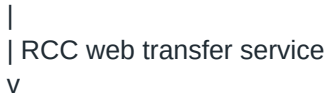
hostname reports the computer currently running the shell.

pwd reports the current working directory.

Run them whenever you are uncertain about the session.

The data-transfer path is separate:

Your computer or an approved data source



Your approved project directory on RCC storage

Do not calculate on the login or submission hosts Do not run analyses, large file conversions, compression jobs, software builds, or long-running programs directly in the VS Code terminal. Create a Slurm job and submit it with sbatch.

2. Before you begin

Accounts, authorisation, and software are separate things

An RCC account proves who you are. Project membership determines which data and computing resources you may use. Installed software determines which programs are available. Having an account does not automatically grant access to every project, and installing VS Code does not create an RCC account.

You need:

- an active RCC account;
- an RCC username;
- permission to use an RCC project or group directory;
- a password manager approved by your institution;
- permission to install Visual Studio Code on your computer; and
- approximately 30 minutes for the initial setup.

Data-protection rule

Only data approved for the RCC environment may be transferred. The existing RCC policy states that directly identifying patient data and other personally identifying information must not be uploaded. Data must be de-identified before transfer unless a project-specific approval and technical environment explicitly allow otherwise.

When uncertain, stop and ask your project coordinator or the RCC support contact before uploading the data.

3. Install the required software

Local software versus remote software

Visual Studio Code and the SSH client are installed on your laptop. They provide the interface and secure connection. The Linux shell, Slurm commands, project files, and later scientific programs are on RCC. When VS Code is connected remotely, the window is local but most file operations and terminal commands happen on RCC.

You need two components on your own computer:

an OpenSSH client; and

Visual Studio Code with the Microsoft **Remote - SSH** extension.

3.1 Windows 10 or Windows 11

Check OpenSSH

Open **PowerShell**.

Run:

```
ssh -V
```

A version string means that the SSH client is available.

If PowerShell reports that ssh is unknown, install **OpenSSH Client** through Windows **Optional features**. On a centrally managed computer, contact local IT if you cannot add the feature yourself.

Install Visual Studio Code

Download and install the current stable release of Visual Studio Code from Microsoft.

Start Visual Studio Code.

Open the **Extensions** view.

Search for Remote - SSH.

Install the extension published by Microsoft.

Do not install PuTTY for this tutorial. Visual Studio Code Remote - SSH expects an OpenSSH-compatible client.

3.2 macOS

OpenSSH is included with macOS.

Open **Terminal**.

Run:

```
ssh -V
```

Download and install the current stable release of Visual Studio Code from Microsoft.

Start Visual Studio Code.

Open the **Extensions** view.

Search for Remote - SSH.

Install the extension published by Microsoft.

4. Create your RCC SSH key

Why RCC uses SSH

SSH stands for **Secure Shell**. It creates an encrypted connection between your laptop and a remote computer. Encryption prevents other systems on the network from reading the session. SSH also verifies identities in both directions:

RCC verifies **you** using your registered public key; and
you verify the **RCC server** using its host-key fingerprint.

This is why the tutorial discusses both user keys and host fingerprints. They solve different security problems.

SSH is also the transport used by VS Code Remote - SSH. VS Code does not bypass SSH; it builds its remote editing session on top of the same connection that you first test in a terminal.

4.1 What an SSH key is

An SSH key pair consists of two files:

the **private key**, which remains on your computer; and
the **public key**, which is registered with your RCC account.

The private key is an authentication credential. Anyone who obtains it may be able to act as you. Never send the private key by email, upload it to a ticket, place it in cloud storage, commit it to Git, or copy it to the cluster.

The public key is designed to be shared with the RCC account administrator.

A useful model is a challenge-and-response test. The server sends a mathematical challenge that can be answered only with the private key. The private key itself is not sent to RCC. The registered public key lets the server verify the answer.

Do not confuse the following credentials:

RCC username: names your account and may be entered in configuration and support requests.

Private SSH key: proves possession of your identity credential and belongs on your laptop only.

Public SSH key: lets RCC verify the private key and is registered with your RCC account.

Key passphrase: encrypts the private-key file and belongs in an approved password manager and your memory only.

Host-key fingerprint: lets you verify the RCC server and must be obtained from official RCC documentation.

4.2 Handling the key passphrase

Use a separate passphrase to protect the private key unless RCC support explicitly instructs you otherwise.

Follow these rules:

Generate the passphrase in your approved password manager.

Save it in the password manager **before** completing key generation.

Do not rely on remembering it.

Do not reuse your RCC, university, or email password.

The key passphrase cannot be recovered or reset. If it is lost, create a new key pair, ask RCC support to register the new public key, and ask them to revoke the old key.

When a terminal asks for the passphrase, no characters or dots are displayed while you type. This is normal.

4.3 Generate the key on Windows

Open PowerShell and run:

```
New-Item -ItemType Directory -Force "$HOME\.ssh" | Out-Null
ssh-keygen -t ed25519 -a 100 -f "$HOME\.ssh\id_rcc" -C "$env:USERNAME@rcc"
```

Enter the passphrase stored in your password manager when prompted.

Two files are created:

```
C:\Users\YOUR_WINDOWS_NAME\.ssh\id_rcc
C:\Users\YOUR_WINDOWS_NAME\.ssh\id_rcc.pub
```

id_rcc is the private key. Never share it.

id_rcc.pub is the public key. This is the file that may be shared.

Copy the public key to the clipboard:

```
Get-Content "$HOME\.ssh\id_rcc.pub" | Set-Clipboard
```

4.4 Generate the key on macOS

Open Terminal and run:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -a 100 -f ~/.ssh/id_rcc -C "$USER@rcc"
```

Enter the passphrase stored in your password manager when prompted.

Two files are created:

```
~/.ssh/id_rcc
~/.ssh/id_rcc.pub
```

id_rcc is the private key. Never share it.

id_rcc.pub is the public key. This is the file that may be shared.

Copy the public key to the clipboard:

```
pbcopy < ~/.ssh/id_rcc.pub
```

4.5 Register the public key

Send only the contents of id_rcc.pub through the approved RCC account-enrolment process. Include the identifying information requested by the account form or project coordinator.

A public key begins with text similar to:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... username@rcc
```

A private key begins with text similar to:

```
-----BEGIN OPENSSH PRIVATE KEY-----
```

Stop if you see PRIVATE KEY Never send a file or text containing BEGIN OPENSSH PRIVATE KEY.

Wait until your project coordinator or RCC support confirms that the public key has been activated.

5. Configure the SSH connection

Why use an SSH configuration file

An SSH configuration file is a saved connection recipe. It records the official host names, your RCC username, the private key to use, and the route through the gateway. This reduces typing and prevents VS Code, the terminal, and later tools from using different connection settings.

ProxyJump means that SSH first connects to the gateway and then opens the connection to the submission host through it. Your terminal still ends on the submission host; the gateway is an intermediate security boundary.

The SSH configuration gives the two RCC systems short, unambiguous names. It also tells SSH to reach the submission host through the login gateway.

Replace <RCC_USERNAME> with the username assigned to you. Do not include the angle brackets.

5.1 Windows SSH configuration

Open PowerShell and run:

```
New-Item -ItemType Directory -Force "$HOME\.ssh" | Out-Null
New-Item -ItemType File -Force "$HOME\.ssh\config" | Out-Null
notepad "$HOME\.ssh\config"
```

Insert the following configuration and save the file:

```
Host rcc-login
  HostName login1.ikim.uk-essen.de
  User <RCC_USERNAME>
  IdentityFile ~/.ssh/id_rcc
  IdentitiesOnly yes
```

```
Host rcc
  HostName shellhost.ikim.uk-essen.de
  User <RCC_USERNAME>
  IdentityFile ~/.ssh/id_rcc
  IdentitiesOnly yes
  ProxyJump rcc-login
  ServerAliveInterval 60
  ServerAliveCountMax 3
```

Confirm that the file is named exactly config, not config.txt.

5.2 macOS SSH configuration

Open Terminal and run:

```
touch ~/.ssh/config
chmod 600 ~/.ssh/config
open -e ~/.ssh/config
```

Insert the following configuration and save the file:

```
Host rcc-login
  HostName login1.ikim.uk-essen.de
  User <RCC_USERNAME>
  IdentityFile ~/.ssh/id_rcc
  IdentitiesOnly yes
  AddKeysToAgent yes
  UseKeychain yes
```

```
Host rcc
  HostName shellhost.ikim.uk-essen.de
  User <RCC_USERNAME>
  IdentityFile ~/.ssh/id_rcc
  IdentitiesOnly yes
  ProxyJump rcc-login
  ServerAliveInterval 60
  ServerAliveCountMax 3
  AddKeysToAgent yes
  UseKeychain yes
```

Add the key to the macOS keychain:

```
ssh-add --apple-use-keychain ~/.ssh/id_rcc
```

The keychain stores the passphrase locally so that you do not need to re-enter it for every connection.

6. Test the SSH connection in a terminal

Why test outside VS Code first

A terminal test separates SSH problems from VS Code problems. If `ssh rcc` fails in PowerShell or Terminal, VS Code cannot repair the underlying account, key, host-name, VPN, or fingerprint issue. If the terminal test succeeds but VS Code fails, the problem is limited to the editor or extension.

An SSH session is a network connection to a remote shell. Commands typed after the connection run remotely until you type `exit`, close the terminal, or lose the connection.

Test the terminal connection before attempting to use Visual Studio Code.

On Windows, use PowerShell. On macOS, use Terminal.

```
ssh rcc
```

6.1 Verify the host fingerprints

User authentication and server authentication are different

Your public key tells RCC which user is connecting. The server fingerprint tells you which server answered. Accepting an unverified fingerprint would protect the traffic cryptographically but could protect a connection to the wrong system. Therefore, compare the fingerprint before accepting it.

During the first connection, SSH may ask you to verify the login gateway and the submission host.

Compare every displayed fingerprint with the values published at the top of this tutorial. Type `yes` only when they match exactly.

If a fingerprint is missing, unexpected, or different:

answer `no` or press `Ctrl+C`;

do not delete entries from `known_hosts`; and

contact RCC support.

A changed host key can be caused by legitimate maintenance, but it can also indicate that the connection is not reaching the intended server.

6.2 Confirm the remote session

After login, run:

```
whoami
hostname
pwd
```

Expected results:

whoami displays your RCC username;

hostname identifies an RCC submission host; and

pwd displays your RCC home directory.

Then disconnect:

```
exit
```

If login fails, collect diagnostic output:

```
ssh -vvv rcc
```

Send the output to RCC support, but never send your private key or its passphrase.

7. Connect with Visual Studio Code

What an IDE is

An integrated development environment, or IDE, combines a text editor, file browser, terminal, search tools, version-control support, and extensions. VS Code is the preferred beginner IDE for RCC because it provides these tools in one window. It does not replace Linux, SSH, Slurm, or the need to understand file locations.

What “Remote - SSH” changes

After connection:

the VS Code window and keyboard input remain on your laptop;

the opened folder is on RCC storage;

the integrated terminal runs on the RCC submission host;

remote extensions may run on RCC; and

saving a file changes the remote file immediately.

This local/remote split is important. Dragging a local file into a remote editor or selecting the wrong VS Code window can produce unexpected copies. Use the approved transfer service for research data.

Visual Studio Code runs on your laptop. The Remote - SSH extension opens a remote workspace on the RCC submission host. Files opened in that remote window are stored on the cluster, and terminals opened there run on the submission host.

7.1 Open the RCC connection

Start Visual Studio Code.

Press F1 to open the Command Palette.

Select **Remote-SSH: Connect to Host...**

Select rcc.

If asked for the remote platform, select **Linux**.

Verify any host-key prompt against the fingerprints published by RCC.

Enter the SSH-key passphrase if requested.

When connected, the lower-left corner of the VS Code window should display an SSH connection such as SSH: rcc.

7.2 Open a terminal on the submission host

Select **Terminal > New Terminal** and run:

```
whoami
hostname
pwd
```

These commands now execute on the RCC submission host, not on your laptop.

7.3 Create the tutorial workspace

In the VS Code terminal, run:

```
mkdir -p ~/rcc-introduction
cd ~/rcc-introduction
```

Then select **File > Open Folder...** and open:

```
~/rcc-introduction
```

This small tutorial directory may reside in your home directory. Real project data and shared analysis files must be stored in the project or group directory assigned to you.

7.4 What may run in the VS Code terminal

A terminal command starts a process immediately

When you type a program name in the VS Code terminal, the shell normally starts that process on the submission host. Slurm is involved only when you explicitly use `sbatch`, `srun`, or a supported workflow profile. A remote terminal is therefore not automatically a compute allocation.

Appropriate commands on the submission host include:

- `cd`, `pwd`, `ls`, and `mkdir`;
- editing scripts and configuration files;
- `sbatch`, `squeue`, `sacct`, and `scancel`;
- small checks of file names and permissions; and
- inspecting short log files.

Do not directly run scientific analyses, large data-processing commands, compression jobs, or long-running programs in this terminal. Submit them to Slurm.

8. Transfer a test file with the RCC web transfer service

Why data transfer is a separate workflow

SSH is optimised for interactive control. VS Code is optimised for editing. Large or restartable research-data transfers need a service that can queue work, retry failures, report individual files, and move data without keeping an editor window open. For that reason, this tutorial uses the RCC web transfer service rather than the VS Code file explorer.

A transfer normally **copies** data. It does not prove that the destination copy is complete and it does not automatically delete the source. Keep the source until verification has succeeded.

Understand the main storage locations

Exact RCC paths and retention policies must be published by administrators, but most clusters distinguish:

Home directory: for small personal configuration and scripts. Do not use it as permanent bulk project storage.

Project or group storage: for shared approved inputs and outputs. Use the exact path assigned to the project.

Scratch or temporary storage: for high-speed intermediate files with limited retention. Use it only when the project documents staging and cleanup.

Laptop storage: for local working copies and transfer sources. Do not keep the only copy of important research data there.

A path beginning with / is an absolute Linux path. A path beginning with ~ is relative to your RCC home directory. Windows drive-letter paths such as C:\ do not exist in the remote Linux shell.

What a checksum proves

A SHA-256 checksum is a compact value calculated from every byte in a file. Matching local and remote checksums provide strong evidence that the file content is identical. A checksum does not prove that the file is scientifically correct, permitted, or placed in the correct project.

This section assumes that browser-based upload and download are enabled for the RCC transfer collection.

8.1 Create a small local test file

Create a plain-text file on your laptop named:

rcc-transfer-test.txt

Add one line of text, for example:

RCC transfer test

Do not use sensitive or patient-related data for the test.

8.2 Identify the destination directory

Use the exact project work directory supplied by your project coordinator. It should normally be located under a project or group path, not in your home directory.

Example only:

/projects/<PROJECT_NAME>/<YOUR_ASSIGNED_DIRECTORY>/incoming

Do not copy this example literally. Project layouts differ.

You can create the assigned incoming directory from the VS Code terminal if you have permission:

```
mkdir -p <YOUR_ASSIGNED_WORK_DIRECTORY>/incoming
```

8.3 Upload through the browser

Open [ADMIN: RCC web transfer address] in your browser.

Sign in using [ADMIN: identity provider and login instructions].

Select the collection [ADMIN: exact RCC collection name].

Navigate to your assigned project directory.

Open the incoming subdirectory.

Select **Upload**.

Choose rcc-transfer-test.txt from your laptop.

Start the transfer.

Wait until the transfer status reports success.

Do not select a destination based only on a similar-looking project name. Use the exact path provided to you.

8.4 Verify the uploaded file on RCC

In the VS Code terminal, run:

```
cd <YOUR_ASSIGNED_WORK_DIRECTORY>/incoming
ls -lah
file rcc-transfer-test.txt
sha256sum rcc-transfer-test.txt
```

Calculate the checksum of the local file as well.

On Windows PowerShell:

```
Get-FileHash .\rcc-transfer-test.txt -Algorithm SHA256
```

On macOS Terminal:

```
shasum -a 256 rcc-transfer-test.txt
```

The local and RCC SHA-256 values must be identical.

For a larger directory, also compare the number of files and review the transfer service's failure report. A successful top-level transfer does not remove the need to notice skipped or failed files.

8.5 Transfer rules

Transfer research data only into an approved project or group directory.

Do not use the RCC home directory as permanent project storage.

Do not transfer a large dataset through the VS Code file explorer.

Do not upload private SSH keys, credentials, password exports, or unapproved identifying data.

Keep the original data until the transfer and integrity checks have completed.

9. Submit your first Slurm job

Why a resource manager is necessary

Many users share a finite number of compute nodes. If everyone started programs on arbitrary nodes, jobs would compete unpredictably for CPU time and memory, systems could become unresponsive, and no fair ordering would exist. Slurm records resource requests, applies site policy, finds a suitable node, starts the job, and records its outcome.

Slurm is the resource manager for the RCC compute cluster. You describe the resources a program needs and submit a batch script. Slurm places the job in a queue and starts it on an appropriate compute node when resources are available.

Batch work versus interactive work

A **batch job** is described in a script and can run without a person watching the terminal. This is the default for reproducible analysis. An **interactive allocation** gives a temporary shell on a compute node and is useful for debugging or software that genuinely requires interaction. Interactive allocations still require Slurm and will be covered separately in advanced documentation.

The job lifecycle

script prepared -> submitted -> pending -> running -> completed or failed

```

      |       |       |
  job ID  compute  logs and
        node    exit status

```

Pending is a normal state. It means Slurm has accepted the job but has not yet found the resources and policy conditions needed to start it.

This first job uses only standard Linux tools. It creates five numbers and calculates their mean. No additional software is required.

What a batch script contains

A batch script has two roles:

#SBATCH directives describe the requested resources and log files to Slurm.

Shell commands describe the work to execute after Slurm starts the job.

Important shell concepts in the example:

#!/usr/bin/env bash selects Bash as the interpreter.

Lines beginning with # are comments, except that Slurm reads #SBATCH lines before execution.

set -euo pipefail makes common scripting errors stop the job instead of silently continuing.

> redirects command output into a file.

\${USER} and \${SLURM_JOB_ID} are environment variables.

A program returns an exit code; zero normally means success, and a non-zero code means failure.

CPU, memory, and time are different resources

CPU cores determine how many instructions can run in parallel when the software supports parallel execution.

Memory holds active data and program state. More CPU cores do not automatically provide enough memory.

Wall time is the maximum elapsed time Slurm allows the job to run.

Requesting far more than needed can delay the job and waste shared capacity. Requesting too little can cause termination or poor performance. Part 2 explains how to use measured resource usage to improve requests.

9.1 Create the directories

In the VS Code terminal:

```
cd ~/rcc-introduction
mkdir -p scripts logs results
```

The logs directory must exist before submitting the job because Slurm opens the log files before the script begins.

9.2 Create the batch script

In the VS Code Explorer, create:

scripts/first-job.sh

Insert the following content:

```
#!/usr/bin/env bash
#SBATCH --job-name=rcc-first
#SBATCH --time=00:02:00
#SBATCH --cpus-per-task=1
#SBATCH --mem=256M
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err
```

```
set -euo pipefail
```

```
echo "Hello from an RCC compute node."
echo "User: ${USER}"
echo "Slurm job ID: ${SLURM_JOB_ID}"
echo "Compute node: $(hostname)"
echo "Started: $(date --iso-8601=seconds)"

printf "value\n1\n2\n3\n4\n5\n" > results/numbers.csv
```

```
awk '
  NR > 1 {
    sum += $1
    n += 1
  }
  END {
    printf "n=%d\nmean=%.2f\n", n, sum / n
  }
' results/numbers.csv > results/summary.txt
```

```
sleep 10
```

```
echo "Finished: $(date --iso-8601=seconds)"
```

Save the file.

The lines beginning with #SBATCH request:

a job name;

a maximum runtime of two minutes;

one CPU core;

256 MB of memory; and

separate output and error log files.

9.3 Submit the job

Make sure the terminal is in the tutorial directory:

```
cd ~/rcc-introduction
pwd
```

Submit the script:

```
sbatch scripts/first-job.sh
```

Slurm responds with a job ID similar to:

Submitted batch job 12345

Write down your job ID.

9.4 Monitor the job

Display your queued and running jobs:

```
squeue -u "$USER"
```

Common job states include:

PD: Pending: the job is waiting for resources or another condition

R: Running

CG: Completing

A short job may disappear from squeue quickly after it finishes. This is normal.

To inspect a completed job, replace <JOB_ID>:

```
sacct -j <JOB_ID> --format=JobID,JobName,State,Elapsed,AllocCPUS,ExitCode
```

9.5 Inspect the result

List the files:

```
ls -lah logs results
```

Display the statistical result:

```
cat results/summary.txt
```

Expected output:

```
n=5
mean=3.00
```

Display the Slurm output log, replacing <JOB_ID>:

```
cat logs/rcc-first-<JOB_ID>.out
```

The log should identify a compute node. It should not identify the submission host on which VS Code is connected.

Check the error log:

```
cat logs/rcc-first-<JOB_ID>.err
```

For a successful job, the error file should be empty.

9.6 Cancel a job when necessary

To cancel a queued or running job:

```
scancel <JOB_ID>
```

Cancel jobs that you submitted by mistake or no longer need.

10. Connections, long-running control processes, and tmux

Why an SSH session can disappear

Your remote shell is reached through a network connection. Closing the laptop, changing networks, a VPN interruption, or a timeout can close that connection. A program started directly in the shell may receive a hang-up signal and stop.

A Slurm batch job is different: after sbatch accepts it, Slurm owns the job. The job continues even when VS Code and SSH disconnect. You can reconnect later and inspect it with squeue or sacct.

What tmux is

tmux is a terminal multiplexer. It creates a shell session on the remote host that can be detached from one SSH connection and reattached from another. It is useful for a long-running **control process**, such as a Snakemake driver that is submitting and monitoring Slurm jobs.

laptop and SSH connection

```
|
v
```

tmux session on submission host

```
|
v
```

workflow driver -> Slurm jobs -> compute nodes

tmux does **not** allocate CPU or memory and does not make it acceptable to run analyses on the submission host. It protects the terminal session, not the scientific computation.

Basic commands, for later use:

```
tmux new -s my-workflow
tmux ls
tmux attach -t my-workflow
```

Detach by pressing Ctrl+B, releasing the keys, and then pressing D. Part 2 uses this pattern for Snakemake. tmux is not required for the short first job in Part 1.

11. Troubleshooting

Use a layered troubleshooting model

Classify the failure before changing configuration:

Local tool layer: Are ssh and VS Code installed?

Network layer: Is the network or VPN route available?

Authentication layer: Is the correct user key registered and readable?

Server-identity layer: Does the fingerprint match?

Remote-session layer: Did you reach the correct submission host?

Storage layer: Does the path exist and do permissions allow access?

Scheduler layer: Did Slurm accept the request, and what state or reason does it report?

Application layer: Did the program itself exit successfully and produce valid results?

Avoid deleting configuration or repeatedly retrying until you know which layer failed.

Permission denied (publickey)

Check:

that the RCC username in `~/.ssh/config` is correct;

that `IdentityFile` points to `~/.ssh/id_rcc`;

that you sent `id_rcc.pub`, not another public key;

that RCC support confirmed activation; and

that the private key still exists on the same computer.

Run `ssh -vvv rcc` and send the diagnostic output to RCC support if the problem remains.

Could not resolve hostname

Check the host names in the SSH configuration. Confirm whether VPN is required. Do not replace the official host names with addresses found in old emails or screenshots.

REMOTE HOST IDENTIFICATION HAS CHANGED

Stop. Do not automatically run `ssh-keygen -R` and do not delete `known_hosts`. Compare the new fingerprint with an official RCC announcement or contact support.

VS Code repeatedly asks for the key passphrase

First confirm that `ssh rcc` works in PowerShell or Terminal. On macOS, run:

```
ssh-add --apple-use-keychain ~/.ssh/id_rcc
```

On a managed Windows computer, ask local IT or RCC support whether the Windows ssh-agent service may be enabled. Do not remove the passphrase merely to suppress repeated prompts.

VS Code connects, but sbatch is not found

You are probably connected to the wrong host. The VS Code remote target for this tutorial must be rcc, which resolves to the Slurm submission host through the login gateway.

The job remains in PD

A pending job is not necessarily an error. Slurm may be waiting for resources. Inspect the reason:

```
squeue -j <JOB_ID> -o "%.18i %.9T %.30R"
```

If the job remains pending unexpectedly, send the job ID and the displayed reason to RCC support.

The Slurm log file is missing

Confirm that you submitted the job from `~/rcc-introduction` and that the logs directory existed before submission.

The browser transfer reports a failure

Do not repeatedly restart a large transfer without reviewing the reported cause. Check the destination path, permissions, available storage, and whether the data type is permitted. Send the transfer task identifier to RCC support if available.

12. Completion checklist

You have completed Part 1 when all of the following are true:

- ☐ You know which file is your private SSH key and have never shared it.
- ☐ Your key passphrase is stored in an approved password manager.
- ☐ `ssh rcc` works from PowerShell or Terminal.
- ☐ You verified the RCC host fingerprints.
- ☐ Visual Studio Code displays SSH: rcc.
- ☐ You can open a remote terminal and identify the submission host.
- ☐ You transferred `rcc-transfer-test.txt` into your approved directory.
- ☐ The local and remote SHA-256 checksums match.
- ☐ Slurm accepted your batch script.
- ☐ `results/summary.txt` contains `mean=3.00`.
- ☐ The job log shows that the script ran on a compute node.

13. What comes next

The RCC onboarding series continues with:

Part 2: reproducible workflows with Miniforge, Bioconda, Snakemake, statistics, and a synthetic DNA example;

Part 3: CPU, GPU, memory, storage, I/O, bottleneck diagnosis, and node-local scratch; and

Part 4: Apptainer containers and containerised Snakemake workflows.

Original Part 2 preview

Part 2 will build on this setup and introduce:

Miniforge and isolated Conda environments;
the Bioconda and conda-forge channels;
a reproducible Snakemake project structure;
Snakemake execution through Slurm;
a small statistical analysis with tables and figures;
a minimal DNA sequence-analysis workflow; and
resource estimation, logs, provenance, and reproducibility.

Do not install a separate Anaconda distribution or create an unstructured collection of environments in preparation. Follow the RCC Miniforge instructions in Part 2 so that all users start from the same supported configuration.